

## LABORATORY RECORD

|              |                            |          |              |
|--------------|----------------------------|----------|--------------|
| Course       | Full Stack Web Development | Date     | 22-09-2026   |
| Course Code  | 23CM4121                   | Exp. No. | 3            |
| Student Name | Badana Pranathi            | Roll No. | A24126552071 |

### 1. TITLE

Design and Implementation of a Student Profile Card Using JavaScript ES6 Classes, DOM Manipulation and Event Handling

### 2. AIM

To develop a Student Profile web application that captures student details through form inputs, encapsulates them using an ES6 class, validates the inputs, and dynamically renders a profile card on the page using JavaScript DOM manipulation, styled with CSS.

### 3. OBJECTIVE

The objectives of this experiment are:

- To design a form with text and number input fields for capturing student details.
- To define an ES6 class Student with a constructor that encapsulates name, roll number, department and CGPA.
- To attach a click event listener to a button using addEventListener.
- To validate that all form fields are filled before processing, using conditional checks and alert().
- To dynamically create and style DOM elements using createElement, textContent and appendChild.
- To prevent duplicate profile cards by clearing previous content with innerHTML before appending new content.
- To style the page using CSS Flexbox, box-shadow and border-radius for a card-based UI.

### 4. THEORY

JavaScript ES6 introduced the class syntax, which provides a cleaner, object-oriented way to create constructor functions and encapsulate related data. The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify at runtime. Methods such as getElementById(), createElement(), appendChild() and textContent allow a script to select existing elements, build new ones, and insert them into the page dynamically. Event listeners such as addEventListener('click', ...) let a page respond to user actions, and simple conditional checks provide client-side form validation before data is processed.

*Application Flow: HTML Form -> Event Listener on Button Click -> Read & Validate Input Values -> Create Student Object -> Build DOM Elements -> Append Profile Card to Page*

### 5. ALGORITHM / PROCEDURE

1. Create index.html with a container div holding four input fields (Name, Roll Number, Department, CGPA) and a 'Show Profile' button, and link style.css and script.js.
2. In script.js, define a class Student with a constructor that stores name, rollNumber, department and cgpa as properties.
3. Attach a click event listener to the button (#showBtn) using addEventListener.
4. Inside the handler, read the value of each input field using getElementById(id).value.
5. Check whether any field is empty; if so, show an alert('Please fill in all fields.') and return from the handler.
6. If all fields are filled, create a new Student object using the collected values.
7. Dynamically create a div (profile card), an h2 heading, and four p elements using document.createElement, and set their text using textContent.
8. Append the heading and paragraphs to the profile div, then clear #profileDisplay with innerHTML = "" and append the new profile div to it.
9. Style the page in style.css using Flexbox centring, rounded corners, box-shadow, and a hover effect on the button.
10. Test the page in a browser: submit with empty fields, then submit with valid data, and verify the rendered profile card.

## 6. PROGRAM

### index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Profile</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h1>Student Profile</h1>

    <input type="text" id="name" placeholder="Enter Name">
    <input type="text" id="roll" placeholder="Enter Roll Number">
    <input type="text" id="department" placeholder="Enter Department">
    <input type="number" id="cgpa" placeholder="Enter CGPA">

    <button id="showBtn">Show Profile</button>

    <div id="profileDisplay"></div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

### script.js

```
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }
}

document.getElementById("showBtn").addEventListener("click", function () {

  const name = document.getElementById("name").value;
  const rollNumber = document.getElementById("roll").value;
  const department = document.getElementById("department").value;
  const cgpa = document.getElementById("cgpa").value;

  if (name === "" || rollNumber === "" || department === "" || cgpa === "") {
    alert("Please fill in all fields.");
    return;
  }

  const student = new Student(name, rollNumber, department, cgpa);

  const profile = document.createElement("div");
  profile.className = "profile";

  const heading = document.createElement("h2");
  heading.textContent = "Student Profile";

  const nameText = document.createElement("p");
```

```

nameText.textContent = "Name : " + student.name;

const rollText = document.createElement("p");
rollText.textContent = "Roll No : " + student.rollNumber;

const departmentText = document.createElement("p");
departmentText.textContent = "Department : " + student.department;

const cgpaText = document.createElement("p");
cgpaText.textContent = "CGPA : " + student.cgpa;

profile.appendChild(heading);
profile.appendChild(nameText);
profile.appendChild(rollText);
profile.appendChild(departmentText);
profile.appendChild(cgpaText);

const display = document.getElementById("profileDisplay");

display.innerHTML = "";
display.appendChild(profile);
});

```

### style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.container {
  background-color: white;
  width: 400px;
  padding: 30px;
  text-align: center;
  border-radius: 10px;
  box-shadow: 0 0 10px gray;
}

h1 {
  color: #333;
}

input {
  width: 90%;
  padding: 10px;
  margin: 8px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

button {
  padding: 10px 20px;
  margin-top: 10px;
  background-color: #333;
  color: white;
  border: none;
}

```

```
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #555;
}

.profile {
    margin-top: 25px;
    padding: 20px;
    text-align: left;
    background-color: #f8f8f8;
    border-radius: 8px;
    border-left: 5px solid #333;
}

.profile h2 {
    text-align: center;
    color: #333;
}

.profile p {
    font-size: 16px;
    margin: 10px 0;
}
```

## 7. CODE EXPLANATION

| Code / Syntax                                            | Explanation                                                                                                       |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>class Student { constructor(...) }</code>          | Defines an ES6 class that encapsulates a student's name, roll number, department and CGPA as instance properties. |
| <code>document.getElementById(id)</code>                 | Selects the input fields (name, roll, department, cgpa) and the button by their id attributes.                    |
| <code>addEventListener("click", function)</code>         | Attaches a click event handler to the Show Profile button that runs the display logic.                            |
| <code>.value</code>                                      | Reads the current text entered by the user in each input field.                                                   |
| <code>if (field === "") { alert(...); return; }</code>   | Validates that no field is left empty; shows an alert and exits the handler if validation fails.                  |
| <code>new Student(...)</code>                            | Creates a new instance of the Student class using the values collected from the form.                             |
| <code>document.createElement(tag)</code>                 | Dynamically creates new DOM elements (a div for the card, an h2 heading, and p elements for each detail).         |
| <code>element.textContent = "..."</code>                 | Sets the visible text of each created element, concatenating labels with the student object's properties.         |
| <code>parent.appendChild(child)</code>                   | Attaches each created element to the profile card, then attaches the completed card to the page.                  |
| <code>display.innerHTML = ""</code>                      | Clears any previously displayed profile before appending a new one, preventing duplicate cards.                   |
| <code>display: flex; justify-content; align-items</code> | Centres the profile container both horizontally and vertically within the viewport.                               |
| <code>border-radius, box-shadow</code>                   | Gives the container and profile card rounded corners and a soft drop shadow for a card-style look.                |
| <code>border-left: 5px solid #333</code>                 | Adds an accent border to the profile card to visually separate it from the form.                                  |
| <code>button:hover</code>                                | Darkens the button's background colour when the user hovers over it, providing visual feedback.                   |

## 8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

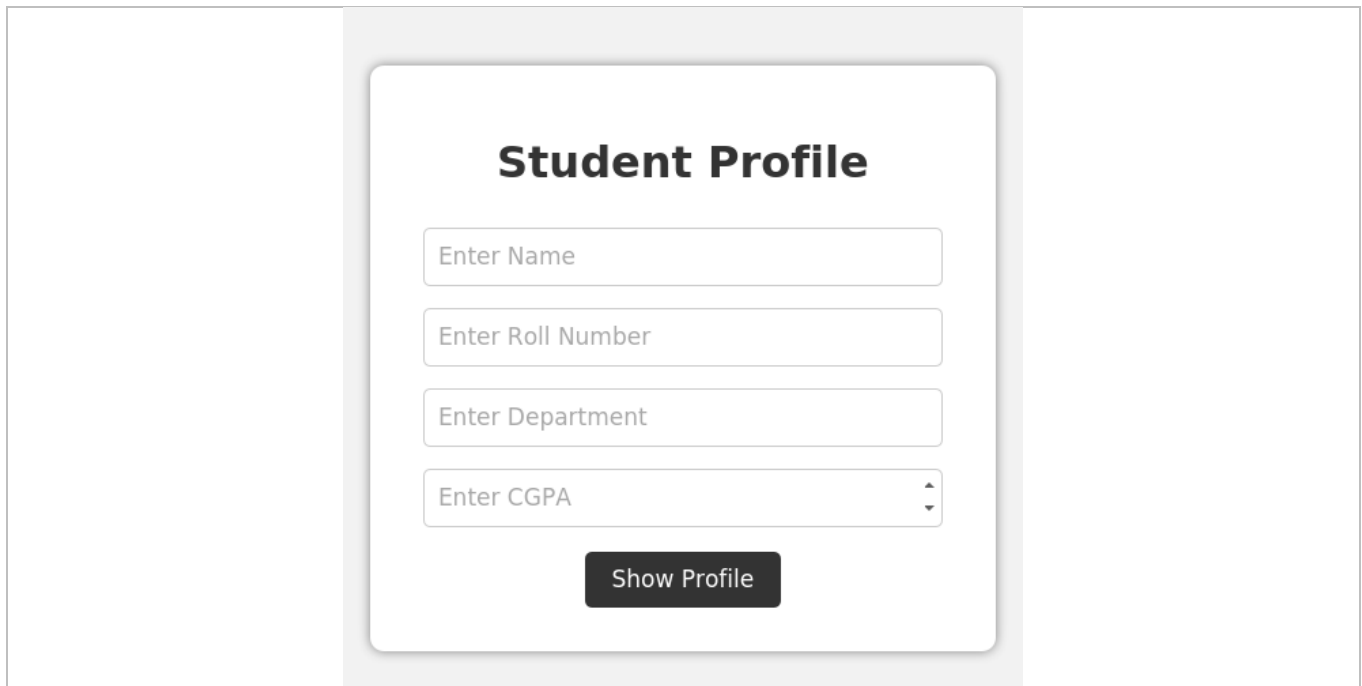
| Test Case | Input / Action                                           | Expected Result                                                              | Actual Result                                     | Status |
|-----------|----------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------|--------|
| TC-01     | Click Show Profile with all fields empty                 | An alert box reads "Please fill in all fields." and no profile card is shown | Alert displayed as expected; profile not rendered | Pass   |
| TC-02     | Click Show Profile with one field (e.g. CGPA) left blank | Alert is triggered since validation checks every field                       | Alert displayed; profile not rendered             | Pass   |

| Test Case | Input / Action                                              | Expected Result                                                                    | Actual Result                                             | Status |
|-----------|-------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------|--------|
| TC-03     | Fill Name, Roll No, Department, CGPA and click Show Profile | A profile card appears showing all four values under a 'Student Profile' heading   | Card rendered correctly with entered values               | Pass   |
| TC-04     | Change field values and click Show Profile again            | The previous profile card is removed and replaced with the new one (no duplicates) | innerHTML reset clears old card before appending new card | Pass   |
| TC-05     | Enter non-numeric text in the CGPA field                    | The number input restricts entry to numeric characters only                        | Field accepts only numeric/decimal input                  | Pass   |
| TC-06     | Hover the mouse over the Show Profile button                | Button background darkens from #333 to #555                                        | Hover effect applied via CSS transition                   | Pass   |

## 9. OUTPUT

### Output: Student Profile Application - Rendered in Browser

#### *Output 1 - Initial Form (Before Submission)*



The screenshot displays a web application titled "Student Profile" centered on a light gray background. The form is contained within a white rounded rectangle with a subtle drop shadow. It features four input fields: "Enter Name", "Enter Roll Number", "Enter Department", and "Enter CGPA". The "Enter CGPA" field includes a small upward and downward arrow icon on its right side. Below these fields is a dark gray button with the text "Show Profile" in white.

#### *Output 2 - Profile Card Displayed After Clicking 'Show Profile'*

## Student Profile

Badana Pranathi

A24126552071

CSE (AI & ML)

9.2

Show Profile

### Student Profile

Name : Badana Pranathi  
Roll No : A24126552071  
Department : CSE (AI & ML)  
CGPA : 9.2

## 10. RESULT

Thus, the Student Profile web application demonstrating JavaScript ES6 classes, DOM manipulation, event handling, and form validation was successfully designed, implemented, and verified.